

COMPETITION SUBMISSION / FOLDER A

APEX-ONE

Computer Programming & Control Architecture

Robo Rumble - Robo Grand Prix

Evidence-based development report

Submission status: Integrated STM32 controller documented from preserved source. Wokwi subsystem demonstrations completed; Pi-to-STM32 UART integration and physical racing performance remain unverified.

Document	Folder A programming and control report
Project	APEX-ONE autonomous racing vehicle
Team	Robo Rumble Team
Version	1.0 - preserved-source documentation
Date	08 September 2026
Primary source	STM32/sketch.ino - original 7,079-line baseline

The report records the present engineering state rather than presenting simulation demonstrations as proof of physical readiness.

01 / REQUIREMENT ALIGNMENT

What Folder A must contain

The competition requires all code needed for the build, an explanation of programming methods and frameworks, and flowcharts that demonstrate category-specific control logic. [G1, pp. 4 and 9]

Assessment area	Required evidence	Where addressed
Framework & flowcharts - 10 points	Clear architecture, methods, frameworks and visual flowcharts.	Sections 2-6; four vector flowcharts.
Code logic & alignment - 10 points	Well-commented code explicitly tailored to autonomous Grand Prix racing.	Sections 4-7; preserved STM32 source and source map.
All required source	Firmware, programming scripts and supporting code.	Source package; development status identifies missing dependencies.

Repository requirements beyond Folder A

The root **README.md** must summarise the project and map the repository. The repository also requires Folder B (Designs) and Folder C (Documentation). Folder B must show the accessible ON/OFF switch and annotated E-Stop power-cut mechanism. [G1, pp. 3-4, 9]

Category constraints

The vehicle is designed for fully autonomous racing: no remote control, Wi-Fi control or human contact during the run. The competition also specifies a maximum 50 x 50 cm footprint and mass strictly below 5 kg. The software documentation does not replace physical compliance evidence.

Scope of this report

This document describes the control implementation, explains its design choices, provides flowcharts and maps the source files. It does not certify mechanical fabrication, electrical power isolation, Unity physics results, physical sensor calibration or a completed real-camera integration.

02 / PROGRAMMING METHODS AND FRAMEWORKS

Two-processor control architecture

APEX-ONE separates planned high-level vision processing from implemented low-level motor and safety control. This is a two-processor architecture, not a claim that the STM32 itself is a dual-core processor.

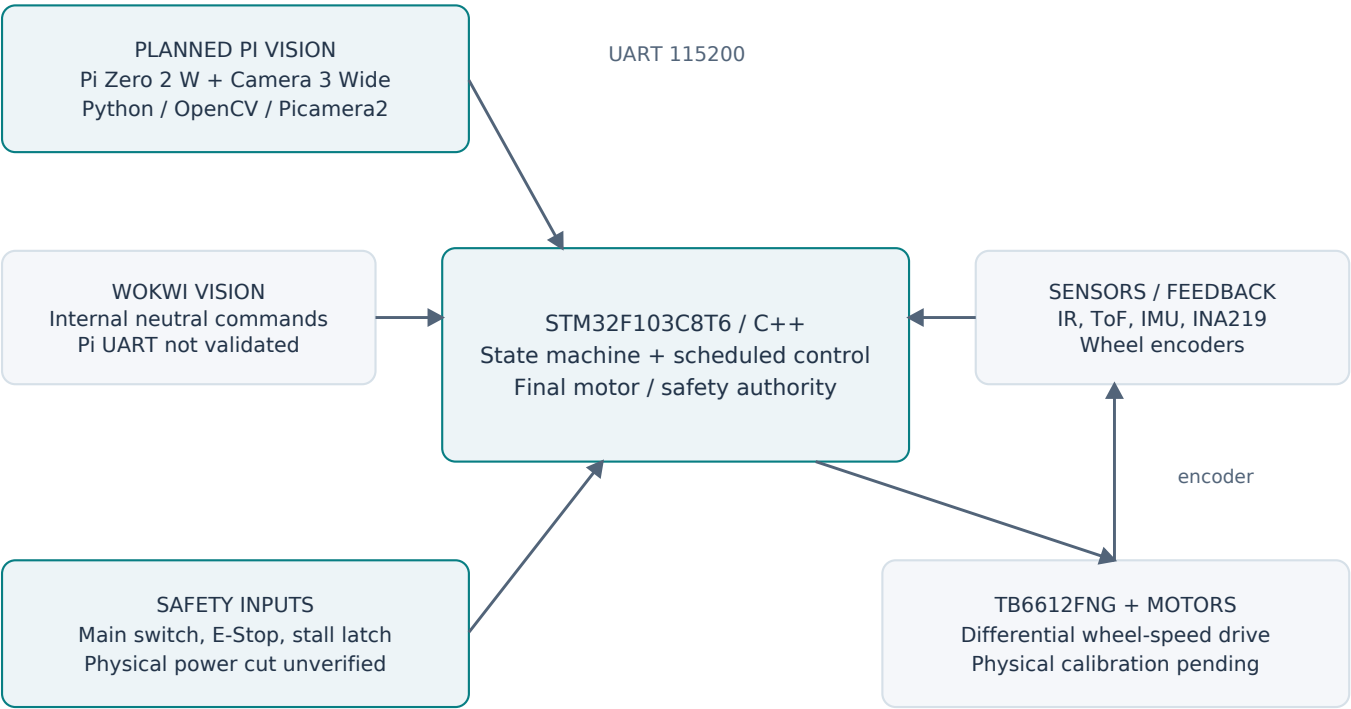


Figure 1. Intended physical architecture and the distinct Wokwi vision path. The Pi UART interface is explicitly marked unvalidated.

Layer	Implementation and role
STM32 / C++	STM32F103C8T6, Arduino-compatible STM32 core, Wire.h, HardwareSerial, GPIO and time-based scheduling. Owns navigation state, motor drive, encoders and safety.
Raspberry Pi / Python	Existing draft uses Picamera2, OpenCV, NumPy and pyserial for road-corridor look-ahead and UART commands. Not validated on the physical Pi.
Wokwi	C++ firmware and custom behavioural sensor models. Internal neutral vision bypasses the failed custom Pi UART link in the preserved baseline.
Unity	Team-reported modelling environment for kinematics and chassis stability. No Unity project, results or reproducible test evidence was supplied for verification here.

03 / PROGRAMMING AND INTERFACES

Methods, frameworks and runtime

The firmware uses a cooperative, time-scheduled control structure rather than an RTOS. It is Arduino-core C++ with hardware-abstraction calls; it should not be described as a purely bare-metal register implementation. [S1]

Method / interface	Actual design and evidence
Cooperative scheduling	millis() intervals gate road control at 10 ms (100 Hz target), PID at 50 ms (20 Hz target), ToF at 50 ms and INA219 at 100 ms. These are requested rates, not measured worst-case deadlines.
Serial communication	DebugSerial(PA10, PA9); PiSerial(PB11, PB10); both configured for 115200 baud. Separate debug and Pi paths prevent intentional port sharing.
I²C bus	Wire on PB7 SDA / PB6 SCL at 100 kHz. Current Wokwi uses custom ToF and INA219 devices; IMU and vision are internally simulated.
Motor output	8-bit duty command; original sketch implements software PWM with a 10 ms period (100 Hz carrier). Physical PWM timing and motor-driver behaviour require validation.
No ROS2 / no RTOS	Neither is used in the preserved firmware. The Python vision draft uses lightweight direct libraries rather than ROS2 middleware.

Vision communication contract

The STM32 parser accepts a newline-terminated ASCII packet with three, four or five numeric fields after **V**. The extended design is:

ASCII UART packet

V,<steer>,<speed>,<confidence>,<bridgeConfidence>,<cornerSeverity>
Example: V,35,80,92,0,50
Steering: -100 left to +100 right; speed: 50-100%; confidence values: 0-100%.

The saved Python draft transmits the shorter three-value format and does not yet send the two additional fields. The original STM32 parser defaults absent bridge/corner values to zero. A versioned interface and real UART test are still required.

04 / CODE ORGANISATION AND CATEGORY ALIGNMENT

How the firmware is structured

The preserved source contains a defined navigation-state enum, independent sensor/feedback routines, target-speed selection and a closed-loop motor controller. The function references below point to the original 7,079-line file. [S1]

Source routine(s)	Purpose / code reference
setup(), loop()	Initialise interfaces, enforce start interlock and repeatedly service the controller. Approx. lines 6493 and 7014.
readMainSwitch(), resetDriveMemoryForRaceStart()	Debounced main switch, one-shot arming and neutral motor/PID memory. Approx. lines 1690-1897.
updateVisionSerial(), parseVisionPacket()	Internal vision generation or UART packet parsing and validation. Approx. lines 2147-2480.
updateToF(), readBoundarySensors(), analyseRoadBoundaries()	Distance/TTC and immediate road-boundary inputs. Approx. lines 2600-2900.
updateMotorStallDetection(), updateSkidDetection()	Current/encoder stall confirmation, latch and stability monitoring. Approx. lines 3115-3300 and 3782.
updateBridgePhase(), determineNavigationState()	Bridge phase tracking and ordered navigation/safety decisions. Approx. lines 3837-4364.
applyNavigationTargets(), updateFastRoadControl()	Translate state into wheel targets; 100 Hz target scheduler. Approx. lines 4522 and 4871.
updatePID(), updateSoftwarePWM()	Encoder feedback, RPM ramp, PID and motor duty generation. Approx. lines 5024 and 1533.
enforceValidNavigationState(), printDiagnostics()	Invalid-state recovery and original detailed telemetry. Approx. lines 5498 and 5636.

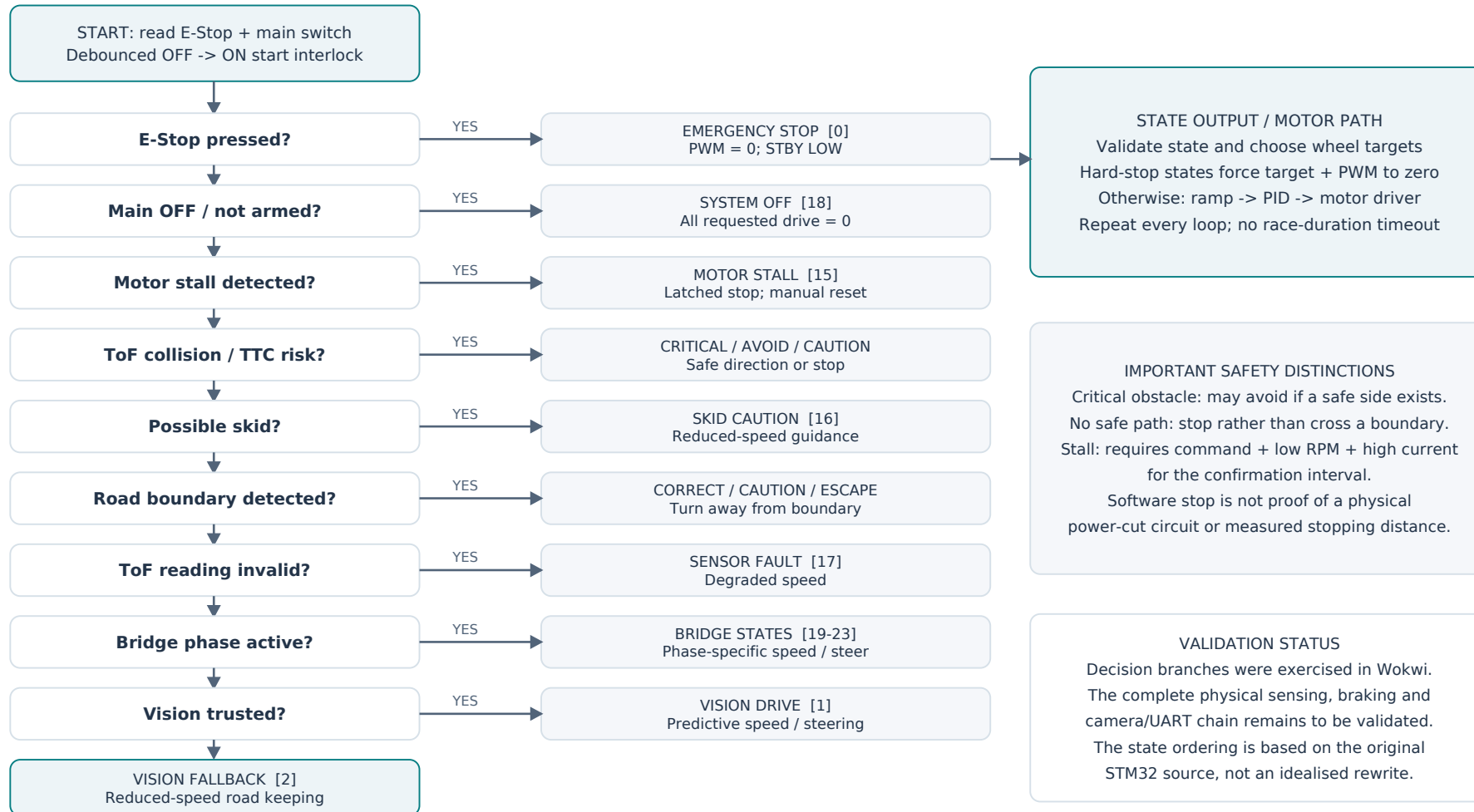
Source preservation

The implementation keeps the original state names, constants, diagnostics and source comments. The PDF is an explanation of the code, not a replacement firmware build. The full, unchanged file and its SHA-256 digest are provided in the source package.

05 / AUTONOMOUS DECISION-MAKING

State-machine priority flowchart

Decision order follows determineNavigationState(); loop() enforces startup before normal control. [S1]



Software motor commands are not proof of a physical emergency power cut.

05 / SYSTEM LOGIC AND STATE MACHINE

State behaviour and safety distinctions

The robot is a road-corridor keeper, not a centre-line follower. IR HIGH means the simulated sensor sees a road boundary. The camera supplies look-ahead guidance; local safety and boundary logic may override it. [S1]

State / codes	Decision and action
0, 18: emergency / system off	Immediate zero drive commands; E-Stop software path also drives STBY LOW. Main OFF requires a fresh start transition.
1-3: vision / fallback / road	Trusted vision provides speed and steering; missing/stale vision falls back to reduced-speed road keeping.
4-7: boundary	Correct left/right, caution or escape from centre/ambiguous boundary. Positive correction turns right.
8-14: collision	Caution, avoidance, critical avoidance, critical stop or no-safe-path stop. Choice is constrained by nearby road boundaries.
15: motor stall	High current + commanded wheel + low measured RPM, confirmed for the configured interval; latch requires main-switch reset.
16-17: skid / sensor fault	Reduce speed on possible skid; degraded mode for invalid ToF data.
19-23: bridge	Approach, climb, crest, descent and exit use phase-specific speed and steering limits.

A stop is not always a power cut

The original critical-stop, no-safe-path and motor-stall paths set requested speed, PID setpoints and PWM to zero. The software E-Stop additionally disables STBY. These are not interchangeable with a verified independent physical power-disconnect circuit. The physical E-Stop must cut motor power as required by the competition, and its actual wiring and stopping behaviour must be demonstrated in Folder B.

Stall decision - exact qualification

A confirmed stall is not instantaneous: the Wokwi threshold is 1800 mA, the wheel must be commanded above 10 RPM and measured below 5 RPM, and the condition is confirmed for 1000 ms. The values are simulation parameters, not validated real-motor protection settings.

06 / MOTION CONTROL

Wheel-speed feedback and speed ramp

Each drive wheel is controlled separately. Differential steering changes the requested left and right RPM; equal requested speeds do not require identical PWM because feedback can differ. [S1]

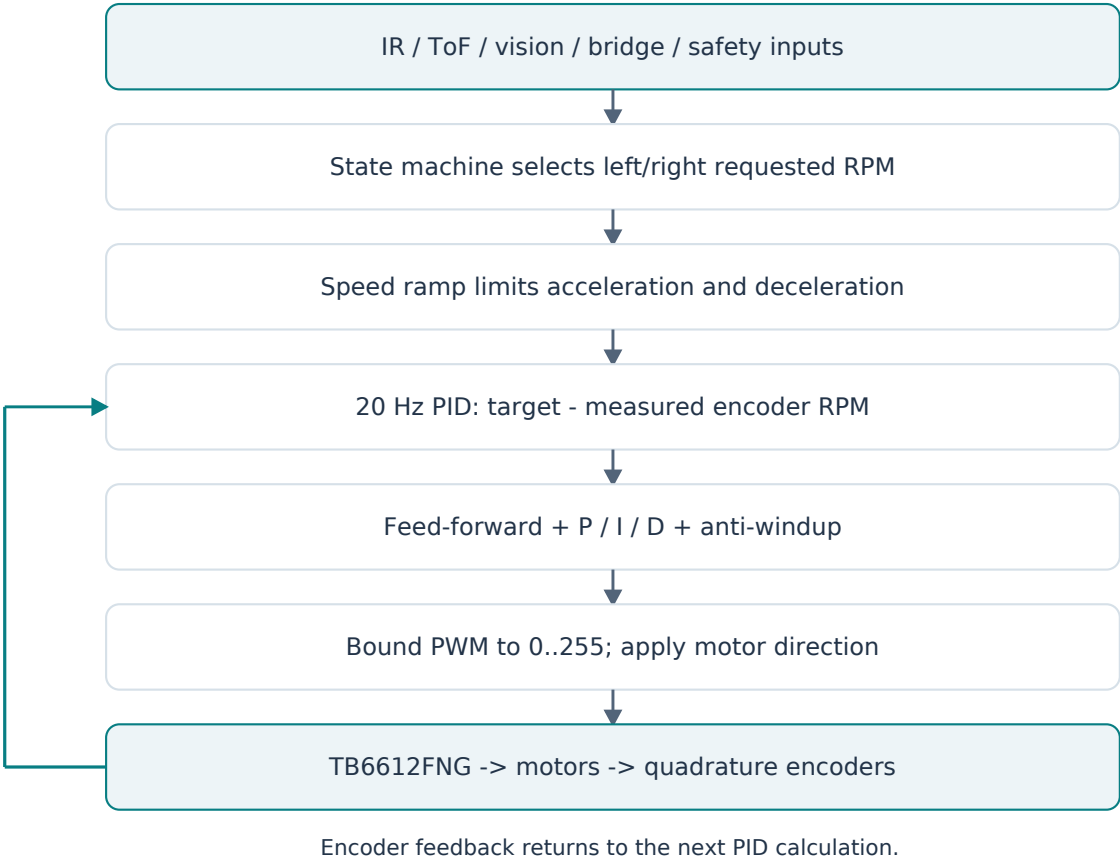


Figure 3. Motor-control data path. The controller uses encoder feedback to trim duty rather than assuming equal motors need equal electrical drive.

Controller detail	Preserved Wokwi value / interpretation
Encoder model	280 counts per wheel revolution (assumed simulation CPR). Actual motor/encoder CPR must be measured.
PID gains	Kp = 2.0; Ki = 0.60; Kd = 0.05. Position-form PID with feed-forward and bounded integral.
Feed-forward	255 / 42.86 PWM per simulated RPM; not a real loaded-motor calibration.
Ramp / sample	30 RPM/s acceleration; 120 RPM/s deceleration; 50 ms PID update. Hard stops bypass the ramp.
RPM measurement	Encoder count change / CPR x 60 / elapsed seconds; filtered RPM is used by PID.

06 / MOTION CONTROL

Corner speed, physical profiles and limits

The camera design estimates future road geometry so speed can be reduced before a corner. The STM32 applies an additional nonlinear cap rather than trusting a speed percentage alone.

Speed policy

Cornering relation: $F = m v^2 / R$
Implemented speed factor: $1 / \sqrt{1 + 2s}$, where s is corner severity from 0 to 1.
Selected base speed: the lower of the camera recommendation and the curvature-based cap, subject to safety limits.

Profile	Intended use and status
Wokwi normal: 38 RPM	Scaled behavioural motor model. It is not the physical car speed.
Physical initial: 600 RPM	Configuration target, about 1.35 m/s with 43 mm wheels. Must be tested under load.
Physical race: 1050 RPM	Proposed straight target, about 2.36 m/s. Not a demonstrated achieved speed.
Near 3 m/s	Longer-term goal only; depends on motor torque, traction, current, road geometry and braking performance.

The original firmware provides different numerical profiles for Wokwi and physical operation, but switching a flag is not sufficient to make the physical system race-ready. The 43 mm wheel and 1500 RPM motor assumptions imply a no-load wheel-speed ceiling of approximately 3.38 m/s, not a guaranteed loaded speed.

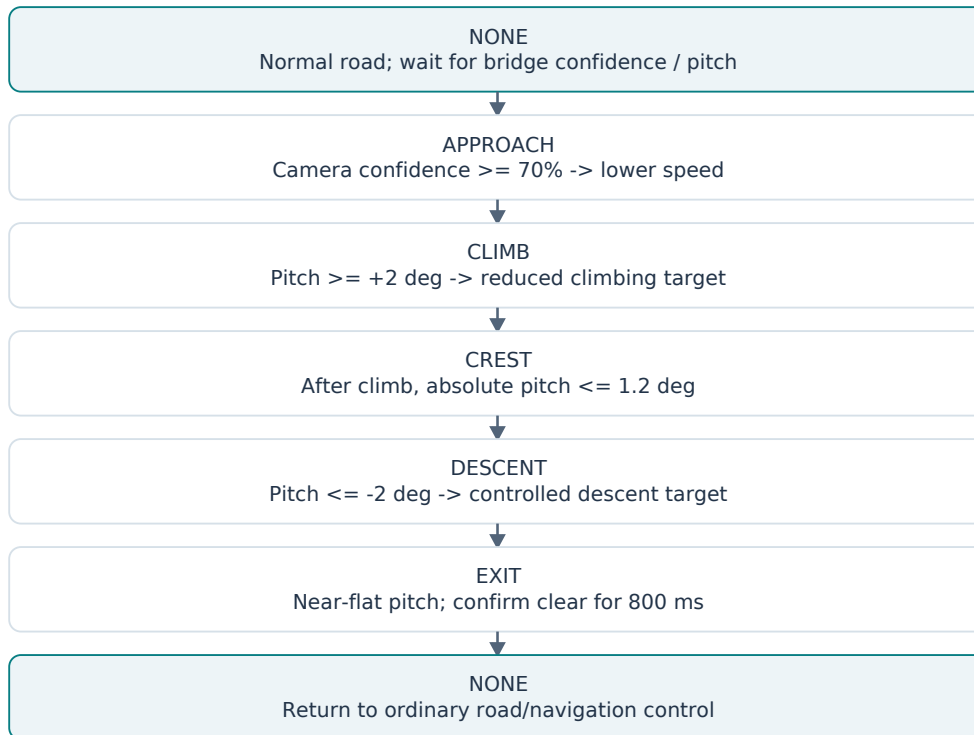
Why we separate the profiles

The Wokwi motor is a simplified behavioural model. Real acceleration, friction, battery voltage, motor torque, gearing, bridge slope and encoder resolution must be measured. The physical PID gains, feed-forward and current thresholds are starting values and must be calibrated before increasing speed.

07 / TRACK-SPECIFIC CONTROL

Bridge handling and crossover awareness

The track includes an overlapping road section implemented as a bridge. The physical vision design must preserve the connected road corridor rather than mistake the lower road for the current path. Bridge logic is present in the STM32; the automatic Wokwi bridge demo is disabled in the preserved baseline.



Physical validation still required

The proposed camera detector must identify the correct road branch, estimate an approaching bridge and send confidence values. The STM32 uses IMU pitch for phase confirmation and can enter climb/descent without camera prediction.

The saved Python draft does not yet transmit bridge confidence or corner severity. Real camera processing, bridge geometry, IMU calibration and safe entry/exit speeds remain unvalidated.

Simulation phases: prior demonstrations exercised approach, climb, crest, descent and exit with temporary pitch/confidence values. That is not proof of physical bridge traversal.

Figure 4. Bridge phase sequence from `updateBridgePhase()`. The original code also supports cancellation of false approach detection, fallback pitch transitions and re-entry to climb/descent when pitch changes.

08 / SIMULATION EVIDENCE

What was demonstrated - and what was not

The test record is a development history, not a claim that every subsystem passed one final end-to-end acceptance run. Earlier versions were changed during debugging. The report therefore records observable outcomes without inventing a final pass rate or runtime.

Test area	Recorded evidence and boundary of the result
Startup and drive	Neutral straight targets, zero PWM while OFF and debounced start behaviour were observed in Wokwi. Later revisions introduced instability; final continuous operation remains unresolved.
IR boundary logic	Left/right correction, centre escape and conflict handling were exercised with explicit simulated IR patterns.
ToF avoidance	Left/right obstacles and an avoidance direction blocked by a boundary produced the expected avoidance or no-safe-path state in development tests.
Corner / bridge	Temporary values exercised nonlinear corner caps and bridge phase transitions. Real image-based detection and physical bridge traversal are not established.
PID / encoders	Ramp, feed-forward/anti-windup and differential wheel-speed response were observed in the behavioural model. Real motor calibration remains pending.
Stall / E-Stop	Per-wheel stall latch and zero PWM shutdown were observed. Physical independent E-Stop power removal was not tested.
Pi UART	Unresolved. STM32 receiver count remained zero in the isolated UART test; full camera-to-motor integration is not validated.

Evidence files should contain the actual screenshots, test settings, date and source revision. The current archive includes a test-evidence index but does not fabricate missing screenshots or laboratory measurements.

09 / SIMULATION LIMITATIONS

Wokwi terminal desynchronisation and interrupted runs

During integration, some Wokwi sessions displayed malformed Serial Monitor text, and other sessions stopped or ceased progressing. The evidence does not establish one definitive root cause. It is therefore not justified to claim that the issue is only a browser text-rendering buffer or that the controller continued flawlessly after every interruption.

Observed behaviour

The development record includes isolated corrupted characters and values such as inf/ovf in printed fields, completely unreadable terminal output, and runs that returned to the Play state. An isolated malformed print may be a rendering or transmission artifact, but a stopped simulation could also involve an error, resource limitation, blocking I²C operation or firmware defect. The available evidence does not distinguish these possibilities conclusively.

Possible contributing factors - not proven causes

The large integrated project combines custom I²C devices, software PWM, encoder polling, high-frequency control and lengthy diagnostics. Earlier custom I²C interactions and Pi UART integration were difficult to stabilise. These factors justify targeted isolation, but they do not prove that Wokwi itself is defective or that all controller calculations are correct.

Physical-hardware distinction

The simulator instability has **not** been observed or reproduced on the physical robot. There is no basis to conclude that the physical STM32, sensors or drivetrain have the same fault. Equally, simulation success is not evidence that the physical robot is fault-free.

Recovery and validation plan

- Preserve the original integrated source and complete Wokwi project snapshot before changing code.
- Run a neutral, main-OFF baseline; record whether the Wokwi timer continues and whether the controller loop advances.
- Isolate the debug UART, each custom I²C device and motor/encoder load using minimal sketches; retain actual error logs.
- Reintroduce subsystems one at a time, record source hashes and compare against the stable baseline.
- Repeat the same functional tests on the actual STM32 with real sensors and measured timing; do not transfer a simulator fault diagnosis to hardware without evidence.

10 / PHYSICAL INTEGRATION PLAN

What changes when custom chips become real

The state-machine and feedback-control architecture can be reused, but the present Wokwi sketch is not a drop-in production firmware image. Actual device initialisation, timing, electrical protection and calibration require dedicated work. [S1-S2]

Subsystem	Required physical integration work
VL53L1X pair	Use actual sensor drivers and ranging readiness/validity checks. Sequence XSHUT and assign unique addresses before enabling both devices.
INA219	Configure real calibration/shunt parameters and use correct signed current scaling. Measure normal, peak and stall currents before setting limits.
MPU6050	Read real measurements; calibrate offsets, mounting orientation and pitch/gyro filtering.
IR array	Confirm actual output polarity, sensor height, road/line thresholds and ambient-light response.
Motors / encoders	Confirm supply voltage, actual CPR, polarity, loaded speed, motor current, encoder sampling and hardware PWM configuration. Retune PID and acceleration/braking limits.
Pi / camera	Deploy and benchmark the Python draft, calibrate the road detector, extend the packet contract and verify UART timeout/recovery end to end.
Power / safety	Implement accessible main power isolation and independent motor-power E-Stop. Verify driver current margin, fuse protection and measured stopping behaviour.

Final racing operation must use the measured physical limits and the competition’s fully autonomous control rule. The Wokwi simulator is an engineering tool, not a substitute for hardware acceptance testing.

11 / SOURCE FILES AND REPRODUCIBILITY

Preserved code and submission inventory

The original controller is the authoritative artifact. Its contents have not been reformatted, repaired, shortened or replaced for this report. Supporting files are labelled by their actual development status.

Repository path	Role / status
STM32/sketch.ino	Original 7,079-line debounced-start integrated controller; unchanged.
Raspberry_Pi/pi_camera_predictive_steering.py	Existing OpenCV/Picamera2 development draft; physical operation unverified.
Raspberry_Pi/README.md	Interface status, planned hardware and remaining work.
Wokwi/diagram.json	Recovered circuit snapshot; not confirmed to be the current browser export.
Wokwi/Custom_Chips/	Available original development snapshots. Exact TB6612FNG and INA219 source/JSON dependencies remain to be exported from the working project.
Flowcharts/*.svg	Four vector diagrams matching the report.
Tests/TEST_EVIDENCE.md	Observed tests, unresolved issues and evidence to collect.
Source_Manifest.json	Source file sizes and SHA-256 checksums.

Original source integrity

SHA-256 of **STM32/sketch.ino**:

Preserved firmware checksum

833b0eeeeaa448512008be18d7f6061a5b5a60469422037d6fa097213915173c7

The latest compacted code was a formatting exercise and is not silently substituted for this original. Before final GitHub submission, the team should export the exact Wokwi project and its custom-chip files, commit that snapshot and record a verified build/test revision.

12 / SUBMISSION READINESS

Remaining actions and reference record

Folder A is ready as a documented development package, but the complete final repository must still be checked against the competition requirements before submission.

Item	Current position / required action
Original STM32 source	Preserved in this package; do not replace it with diagnostic experiments.
All current Wokwi chip files	Incomplete source snapshot. Export missing exact chip files and save the working project URL/export.
Flowcharts and architecture	Provided in this report and as SVG source files.
Pi/OpenCV integration	Draft included; UART and physical camera pipeline remain unresolved.
Unity modelling	Team-reported; attach actual Unity project/results before claiming validation.
Physical safety and speed	Not certified by Folder A; validate in physical build and document in Folder B.
Repository structure	Create/complete root README.md, Folder B and Folder C with their required deliverables.

Source and evidence references

- [G1]** Robo Rumble: Elimination Round Guidelines & Submissions, pp. 3-4 and 9. Competition requirements and scoring rubric.
- [S1]** Original restored STM32 sketch, 7,079 lines; SHA-256 recorded in Section 11 and Source_Manifest.json. Control implementation and configuration.
- [S2]** pi_camera_predictive_steering.py, 544-line development snapshot. Python/OpenCV road-corridor and UART draft; not a verified physical deployment.
- [E1]** Team-provided Wokwi screenshots and development test record. Used only for observed behaviours; no unprovided measurements or pass/fail results are inferred.

Submission statement

The source package is a transparent record of implemented and planned work. It does not claim that the physical robot is completed, that the Pi UART works, or that the cause of Wokwi instability has been proven.